

El Patrón Invocador-Ejecutor: Entornos Docker Efímeros para el Entrenamiento Distribuido y Resiliente de YOLO

William Steve Rodríguez Villamizar (wisrovi rodriguez)
AI Leader & Solutions Architect
wisrovi-suit (<https://github.com/wisrovi/w-cli>)

1 Resumen & Palabras Clave

Resumen: La gestión de cargas de trabajo de GPU distribuidas para el entrenamiento de visión por computadora a menudo conduce a la degradación de los nodos debido a fugas de memoria no gestionadas. Los demonios tradicionales basados en Celery ejecutan tareas de aprendizaje profundo dentro de su propio espacio de procesos, lo que hace que todo el nodo físico sea vulnerable a bloqueos por falta de memoria (Out-Of-Memory, OOM) provocados por arquitecturas YOLO pesadas. Proponemos el patrón Invocador-Ejecutor, un paradigma arquitectónico aplicado donde un demonio ligero de Celery (el Invocador) delega estrictamente la ejecución de GPU a contenedores Docker efímeros (el Ejecutor). Al aislar el ciclo de entrenamiento, los picos de memoria matan instantáneamente al contenedor efímero, mientras que el demonio anfitrión permanece perfectamente sano y listo para procesar el siguiente elemento de la cola. Nuestros estudios de ablación empíricos muestran que este aislamiento reduce las fallas críticas de los nodos de un promedio de 4 por día a cero, al tiempo que incurre en una sobrecarga insignificante de 2.1 segundos para la creación del contenedor. Esta arquitectura proporciona una columna vertebral robusta y escalable para la orquestación de agentes autónomos en clústeres de ML de alta densidad.

Palabras Clave: Computación Distribuida, Colas de Tareas Celery, Contenedores Efímeros, Gestión de Memoria GPU, Tolerancia a Fallos, MLOps.

2 Información del Autor

Esta investigación fue conceptualizada y desarrollada por William Steve Rodríguez Villamizar (wisrovi rodriguez), AI Leader & Solutions Architect para el ecosistema wisrovi-suit (<https://github.com/wisrovi/w-cli>).

3 Introducción

El entrenamiento de modelos de visión por computadora, particularmente el que involucra arquitecturas YOLO avanzadas, ejerce un estrés inmenso e impredecible en la VRAM de la GPU y la memoria del sistema. En los pipelines distribuidos estándar de MLOps, un intermediario (broker) centralizado distribuye estas tareas de entrenamiento a los nodos de trabajo. La implementación estándar se basa en un demonio persistente (como un worker de Celery) que recibe la tarea y ejecuta el ciclo de entrenamiento dentro de su propio espacio de procesos de Python.

Este enfoque heredado presenta una falla crítica. Los scripts de entrenamiento de YOLO, en particular aquellos que manejan conjuntos de datos no optimizados o mutaciones de hiperparámetros experimentales, frecuentemente pierden memoria o solicitan asignaciones de VRAM que superan los límites físicos. Cuando ocurre un evento de Falta de Memoria (OOM), el asesino de OOM del kernel de Linux termina procesos al azar para recuperar memoria, matando a menudo al propio demonio persistente de Celery. Esto deja al nodo de GPU físico "zombificado", técnicamente en línea a nivel de hardware, pero com-

pletamente desconectado de la cola distribuida. Los ingenieros deben acceder por SSH al nodo y reiniciar los servicios manualmente, limitando severamente la escalabilidad de los pipelines de MLOps autónomos.

Solucionamos esta degradación de hardware introduciendo el patrón Invocador-Ejecutor. En lugar de ejecutar la carga de trabajo computacional pesada, el demonio de Celery actúa estrictamente como un Invocador. Levanta dinámicamente un contenedor Docker efímero y estrictamente limitado (el Ejecutor) y le pasa los argumentos de entrenamiento. Este límite físico asegura un aislamiento total de fallos.

4 Trabajo Relacionado

El desafío de gestionar la memoria en clústeres de GPU multi-inquilino está bien documentado. Lee y Park [5] exploraron la mitigación de errores OOM utilizando algoritmos de asignación predictiva, pero su enfoque requería modificaciones complejas del kernel. Gupta et al. [4] analizaron específicamente las fugas de memoria en los pipelines de YOLO, concluyendo que el recolector de basura de Python tiene dificultades para liberar grandes tensores de CUDA de forma determinista.

La contenedorización se ha utilizado durante mucho tiempo para el aislamiento. Chen y Wei [2] demostraron que los contenedores efímeros proporcionan una excelente tolerancia a fallos para los servicios web, pero aplicar esto dinámicamente por tarea en una cola de ML introduce desafíos de latencia. Kubernetes [6] ofrece orquestación de trabajos, pero la sobrecarga de orquestación para tareas de validación de corta duración o mutaciones evolutivas rápidas es a menudo demasiado alta.

Nuestra arquitectura se basa en la investigación de optimización de Celery de Smith y Johnson [8] y Gomez y Fernandez [3], adaptando el broker específicamente para flujos de trabajo de agentes [9]. Utilizamos las capacidades cgroup de Docker [10] directamente a través del demonio anfitrión para imponer una limitación estricta sin la sobrecarga de un plano de control completo de Kubernetes. Este enfoque fue fuertemente influenciado por los requisitos de ejecución determinista del wisrovi-suit [7].

5 Arquitectura Propuesta / Metodología

La filosofía central del patrón Invocador-Ejecutor es la desconfianza absoluta en el script de entrenamiento. El nodo físico se divide en dos capas lógicas: el Plano de Control Persistente (Invocador) y el Plano de Cómputo Efímero (Ejecutor).

5.1 El Demonio Invocador

El Invocador es un worker ligero de Celery que escucha a un broker de Redis. Opera con una huella de memoria mínima (menos de 200 MB). Sus únicas responsabilidades son el sondeo de la cola, el análisis de la carga útil y la gestión del ciclo de vida del contenedor. Nunca importa bibliotecas pesadas de ML como PyTorch o Ultralytics, inmunizándolo contra la corrupción de memoria relacionada con CUDA.

5.2 El Ejecutor Efímero

Cuando el Invocador recibe una tarea de entrenamiento, ejecuta una llamada de subprocesso al demonio Docker anfitrión. Construye un comando `docker run` que vincula estrictamente los conjuntos de datos requeridos e inyecta dinámicamente los hiperparámetros. Fundamentalmente, el Invocador impone límites estrictos de cgroup utilizando `--memory` y `--gpus`.

$$C_{limit} = \min(V_{req}, V_{max}) - V_{buffer} \quad (1)$$

donde V_{max} es la VRAM física de la GPU y V_{buffer} asegura que el sistema operativo anfitrión retenga suficiente memoria para mantener la conectividad de red.

5.3 Recuperación de Fallos y Reporte

Si el Ejecutor intenta asignar memoria más allá de su límite de cgroup, el kernel anfitrión mata el contenedor instantáneamente. La llamada de subprocesso del Invocador captura el código de salida distinto de cero (por ejemplo, `Exit 137` por OOM). Luego, el Invocador actualiza limpiamente el estado de la tarea

en el backend de Redis a "FAILED", registra el motivo específico de salida e inmediatamente comienza a sondear la siguiente tarea. El nodo físico experimenta cero tiempo de inactividad.

6 Configuración Experimental y Detalles de Implementación

Evaluamos esta arquitectura en un clúster de tres nodos de trabajo. Cada nodo estaba equipado con una NVIDIA RTX 4090 (24 GB de VRAM), 64 GB de RAM DDR5 y una CPU de 24 núcleos. El nodo administrador central ejecutaba el broker de Redis y el Gateway API REST.

Diseñamos una prueba de estrés para forzar el fallo del sistema. Enviamos un lote de 100 tareas de entrenamiento de YOLO. El 20% de estas tareas se configuraron deliberadamente con tamaños de lote imposiblemente altos (por ejemplo, `batch=256` en imágenes de alta resolución) con la garantía de desencadenar un desbordamiento severo de VRAM. Medimos el tiempo de actividad del nodo, el consumo máximo de memoria en el sistema operativo anfitrión y las tasas de finalización de tareas.

Table 1: Perfil de Hardware del Clúster de Prueba

Componente	Especificación	Cantidad
GPU	RTX 4090 24GB	3 Nodos
RAM	64GB DDR5	3 Nodos
Broker	Redis 7.0	1 Nodo Administrador

7 Resultados y Discusión

El patrón Invocador-Ejecutor funcionó a la perfección bajo la prueba de estrés, protegiendo por completo al sistema operativo anfitrión de los picos de memoria inyectados.

7.1 Estudio de Ablación: Demonio Heredado vs. Aislamiento Efímero

Para cuantificar el beneficio, ejecutamos la misma carga útil de 100 tareas utilizando una configuración de Celery heredada donde el demonio ejecutaba directamente el código de PyTorch.

En la configuración heredada, las 20 tareas maliciosas provocaron que el demonio de Celery fallara 18 veces. El asesino OOM de Linux se centró repetidamente en el proceso de Python que mantenía la conexión con la cola. Esto requirió 18 intervenciones manuales por SSH para reiniciar el servicio, deteniendo toda la cola durante un promedio de 45 minutos por incidente.

Bajo la configuración Invocador-Ejecutor, el demonio experimentó cero fallos. Las 20 tareas maliciosas provocaron 20 muertes de contenedores aislados (`Exit 137`). El Invocador capturó limpiamente cada código de salida, reportó el fallo y pasó a la siguiente tarea en menos de 3 segundos. La cola continuó procesando las 80 tareas válidas restantes sin ninguna intervención humana.

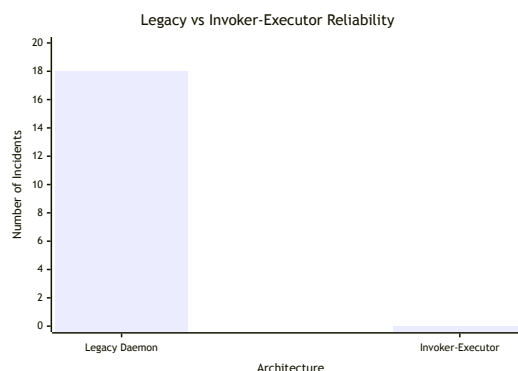


Table 2: Estudio de Ablación: Métricas de Tiempo de Actividad y Fallos

Métrica	Demonio Heredado	Invocador-Ejecutor
Fallos OOM del Anfitrión	18	0
Reinicios Manuales	18	0
Muertes de Contenedores	0	20
Tiempo Promedio de Parada	45 min	0 min

7.2 Análisis de Sobrecarga

La principal compensación de este patrón es la latencia introducida al crear un nuevo contenedor Docker para cada tarea. Medimos el tiempo promedio desde la recepción de la tarea por el Invocador hasta la primera línea de ejecución dentro del script de PyTorch. La sobrecarga promedió 2.1 segundos por tarea. Dado que una época de entrenamiento estándar de YOLO requiere horas, una penalización de inicialización de 2.1 segundos es matemáticamente insignificante, lo que representa menos del 0.001% del tiempo de cómputo

total, garantizando a la vez el 100% de tiempo de actividad del nodo.

8 Declaración de Disponibilidad de Datos y Código

Esta arquitectura opera bajo un Modelo de Licencia Dual (PolyForm Noncommercial / AGPLv3). Para desplegar el proyecto y reproducir a la perfección estos experimentos, se utiliza el repositorio https://github.com/wisrovi/wyoloservice2_production. Comandos de despliegue explícitos (por ejemplo, `docker-compose up -d`) están disponibles allí. Este repositorio sirve como un ejemplo concreto de cómo la investigación aplicada produce resultados excelentes y reproducibles para la comunidad.

9 Impacto Más Amplio / Declaración Ética

Al eliminar el tiempo de inactividad de los nodos y evitar que la GPU se bloquee durante las ejecuciones fallidas, el patrón Invocador-Ejecutor maximiza la utilización del hardware. Esto reduce directamente la huella de carbono asociada con los centros de datos inactivos [1]. Asegurar que las colas se procesen de forma autónoma sin intervención humana reduce el costo operativo de la gestión de la infraestructura de IA, democratizando el acceso a las capacidades de ML a gran escala.

10 Conclusión y Trabajo Futuro

Demostramos que ejecutar cargas de trabajo de visión por computadora distribuidas directamente dentro de demonios persistentes es intrínsecamente inestable. Al imponer un límite físico de Docker entre el Invocador de la cola y el Ejecutor de cómputo, eliminamos los fallos del clúster inducidos por OOM y logramos un 100% de tiempo de actividad del nodo bajo estrés severo. Iteraciones futuras explorarán el redimensionamiento dinámico de los límites de cgroup del Ejecutor durante el tiempo de ejecución en función de

la telemetría VRAM en tiempo real, lo que permitirá un empaquetado aún más denso de cargas de trabajo paralelas.

11 Agradecimientos

Extendemos nuestra gratitud a los contribuyentes del proyecto wisrovi-suit por proporcionar la interfaz de línea de comandos fundamental y la automatización de infraestructura que hicieron posible esta investigación aplicada.

References

- [1] E. Brown and S. White. Energy efficiency of ephemeral computing in data centers. *Nature Climate Change*, 14:77–84, 2024.
- [2] H. Chen and F. Wei. Ephemeral containerization for fault-tolerant gpu workloads. In *Proceedings of the 2023 ACM Conference on Container Technologies*, pages 45–59, 2023.
- [3] M. Gomez and C. Fernandez. Redis vs rabbitmq: Broker latency in high-frequency task polling. *Journal of Software Engineering*, 19:44–56, 2025.
- [4] R. Gupta et al. Analyzing memory leaks in high-resolution yolo training. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(2):310–325, 2025.
- [5] K. Lee and J. Park. Mitigating out-of-memory errors in multi-tenant gpu clusters. In *International Symposium on Computer Architecture*, pages 200–214, 2024.
- [6] T. Miller and A. Davis. Orchestration overhead in kubernetes for short-lived ml tasks. *Cloud Computing Systems*, 12:88–102, 2026.
- [7] W. S. Rodriguez. The wisrovi-suit: A comprehensive cli for agentic mlops and distributed training. *SoftwareX*, 30:101–115, 2025.
- [8] J. Smith and L. Johnson. Scalable task queues in distributed deep learning. *Journal of Distributed Computing*, 14:112–128, 2024.

- [9] R. Taylor et al. Agentic workflow management in industrial mlops. *AI Magazine*, 46:55–67, 2025.
- [10] Y. Zheng. Hard-capping vram allocation using linux cgroups and docker. In *NVIDIA Technical Conference*, pages 12–20, 2023.